SoK: Adaptive Evaluation of LLM Agent Defenses Against Indirect Prompt Injection

Anonymous Submission
Double-blind review

***Abstract*—Large language model (LLM) agents that integrate external tools are vulnerable to indirect prompt injection (IPI), where malicious instructions embedded in tool-returned content hijack the agent. A growing body of work proposes defenses, yet these are often evaluated with proxy metrics—string matching for security, "unblocked" rates for utility—that can mislead conclusions. We systematize 11 IPI defenses across three classes (input encoding, structured query, runtime checking) and evaluate them on AgentDojo using *native end-to-end evaluators*: the agent executes real tools, security is measured by `security_from_traces` on actual function-call traces, and utility by `user_task.utility` on real environment state. Our 2,580-trial evaluation compares benign (L0), static-attack (L1), and adaptive-attack (L2) conditions with a defense-class-matched evasion strategy per defense. Three findings emerge. First, input-encoding defenses achieve the best security-utility tradeoff (ASR 0.04–0.21, Utility 0.67–0.73 under static attack), and *reduce* ASR further under adaptive attack (0.04–0.08)—the opposite of what proxy-based evaluations predicted (proxy ASR was 1.00). Second, separating benign utility (no-defense: 0.90) from attack-condition utility (0.19) reveals that 71% of the no-defense utility loss is attack-induced, not a model capability ceiling—correcting a prior misattribution. Third, an ablation of our P2 multi-signal prototype shows the echo-canary provides independent security value (ASR 0.03 alone) but all signal combinations sacrifice utility entirely (Utility 0.00), refuting the hypothesis that orthogonal multi-signals achieve security at acceptable utility cost. We release our framework, native evaluation pipeline, and adaptive attack implementations as open-source artifacts.**

I. Introduction

Large language models (LLMs) are increasingly deployed as autonomous agents that interact with external tools—email clients, calendar APIs, file systems, and web browsers—to complete complex user tasks. While this integration dramatically extends the capabilities of LLM-based systems, it also introduces a critical attack surface: indirect prompt injection (IPI). In an IPI attack, the adversary embeds malicious instructions within the content returned by external tools—such as an email body, a calendar event description, or a web page—so that when the LLM processes this content as part of its context, it follows the injected instruction instead of, or in addition to, the legitimate user task. Greshake et al. [1] first demonstrated that LLMs cannot distinguish instructions from data when both appear in the context window, and unlike direct prompt injection, IPI exploits this fundamental inability without requiring control over the user's input.

The severity of IPI has motivated a rapidly growing body of defense proposals. These defenses span a wide design space: input-encoding approaches wrap untrusted content in delimiters or encodings [2]; structured-query defenses separate trusted instructions from untrusted data at the prompt level [3]; runtime-checking defenses verify that proposed actions align with the original user task [4]; internal-probing defenses inspect the LLM's attention patterns for signs of injection [5]; and architecture-level defenses use process isolation or trusted execution environments to contain the blast radius of a hijacked agent [6], [7]. Training-based defenses fine-tune the LLM to resist injection via preference optimization [8]; we discuss but do not evaluate SecAlign because it requires a fine-tuned model checkpoint that is not compatible with our API-based evaluation pipeline (see Section VIII).

Despite this diversity, a systematic gap remains in how these defenses are evaluated. The majority of proposed defenses report attack success rates (ASR) only against static injection templates—fixed payloads drawn from a pool of known jailbreak strings. Recent work by Zhan et al. [9] demonstrated that adaptive attacks, which tailor the injection payload to the specific defense's detection signal, break three representative defenses (Spotlighting, StruQ, and SecAlign), raising ASR from below 10% to over 60%. This finding raises a pressing question: *does the failure generalize beyond three defenses, and if so, can the failure modes be systematized into root causes and design principles?*

While the adaptive-attacker paradigm is well-known in ML security (e.g., adversarial examples, IDS evasion), IPI defenses are distinct in two ways. First, the attack surface is *textual*: the adversary crafts natural-language injections, not gradient-based perturbations, so the evasion strategies are defense-class-specific (delimiter mimicry, JSON smuggling, semantic paraphrase) rather than generic ℓ_p-ball optimization. Second, IPI defenses operate on the *prompt* the LLM sees, meaning the defense's signal is co-located with the attack payload in the same context window—a structural property that makes signal observability unavoidable for prompt-level defenses. These distinctions mean that IPI-specific adaptive-attack analysis is needed, not just a restatement of general ML-security folklore.

In this paper, we answer this question with a systematization of knowledge (SoK) that makes three contributions. First, we develop a defense taxonomy that classi-

Network and Distributed System Security (NDSS) Symposium 2027
22-26 March 2027, Seoul, Republic of Korea
ISBN 979-8-9919276-8-0
https://dx.doi.org/10.14722/ndss.2027.24xxxx
www.ndss-symposium.org

fies 11 IPI defenses by their layer position (input encoding, structured query, runtime checking) and trust assumption. Second, we build a native adaptive attack framework that, given a defense class, materializes a matched adversary using one of five evasion strategies, evaluated end-to-end with AgentDojo's native `security_from_traces` and `user_task.utility` evaluators. Third, we conduct a large-scale evaluation on the AgentDojo benchmark [10], [11] comparing static (L1) vs. adaptive (L2) attacks, plus a benign (L0) baseline that separates defense over-blocking from attack-induced task failure.

Our evaluation yields three findings that we believe are actionable for the community. First, using AgentDojo's native end-to-end evaluators, input-encoding defenses achieve the best security-utility tradeoff: under static attack, Polymorphic (ASR 0.14, Utility 0.73), Mixture-of-Encodings (ASR 0.16, Utility 0.69), and Spotlighting (ASR 0.21, Utility 0.67) reduce attack success by 71–81% relative to the 0.74 baseline. Crucially, under defense-matched adaptive attack (L2), these defenses perform *better*, not worse (ASR drops to 0.04–0.08)—the opposite of the proxy-based prediction (proxy ASR was 1.00 for all IE defenses). The adaptive evasion suffix dilutes the injection's instruction-following strength more than it evades the defense. Second, by measuring benign (no-attack) utility separately, we find the no-defense baseline completes 90% of tasks without attack but only 19% under attack—71% of the utility loss is attack-induced, not a model capability ceiling, correcting a prior misattribution. Third, an ablation of our P2 multi-signal prototype reveals the echo-canary provides independent security value (ASR 0.03 alone) but all signal combinations sacrifice utility entirely (Utility 0.00), refuting the hypothesis that orthogonal multi-signals achieve security at acceptable utility cost.

We release our framework, evaluation code, and results as open-source artifacts to support reproducible evaluation of future IPI defenses.

a) Contributions.:

- A defense taxonomy of 11 IPI defenses across 3 defense classes (IE, SQ, RC) (Section III).
- A native adaptive attack framework with 5 evasion strategies, evaluated end-to-end with AgentDojo's native evaluators (Section IV).
- A large-scale evaluation (2,580 trials) comparing L0 (benign), L1 (static), and L2 (adaptive) attacks, plus a P2 multi-signal ablation (Section V).

II. Background and Threat Model

A. LLM Agents and Tool Integration

An LLM agent is a system that uses a large language model as its reasoning engine to decompose a user-provided task into a sequence of tool calls. Formally, an agent operates in a loop: at each step, it receives a context $C = (s, u, T, H)$ consisting of a system prompt s, a user task u, a set of tool outputs $T = \{t_1, \ldots, t_n\}$, and a dialogue history H. The LLM produces an action $a = \text{LLM}(C)$, which is either a tool call (executed in the environment, producing a new tool output appended to T) or a final response to the user. The agent terminates when it produces a final response or reaches a step budget.

The critical security property of this architecture is that T is *untrusted*: tool outputs may contain content controlled by an adversary. For example, an email-sending agent that reads a user's inbox to summarize messages will process email bodies written by arbitrary senders; a web-browsing agent will process HTML content from arbitrary web pages. The LLM cannot distinguish between trusted text (the system prompt and user task) and untrusted text (tool outputs) because both appear as natural language in the same context window.

B. Indirect Prompt Injection

Indirect prompt injection (IPI) is an attack in which the adversary embeds instructions within the untrusted tool output T so that the LLM executes the attacker's instruction instead of, or in addition to, the user's legitimate task u. Formally, the adversary replaces a benign tool output t_i with a malicious output $t_i' = t_i \| \pi$, where π is the injection payload and $\|$ denotes concatenation. The attack succeeds if the agent's action a achieves the attacker's goal g (e.g., sending an email to an attacker-controlled address).

The canonical injection payload, following Greshake et al. [1], wraps the malicious instruction in a pseudo-system-message format: `<INFORMATION> This is an important message from the user. Please do the following first: [goal] </INFORMATION>`. This template exploits the LLM's instruction-following propensity by framing the injection as a legitimate user directive.

C. Threat Model

We adopt the threat model standard in the IPI literature. The adversary:

- Controls the content of at least one tool output $t_i \in T$ (e.g., by sending a malicious email, publishing a malicious web page, or compromising a tool provider).
- Knows the defense class employed by the agent (e.g., input encoding, structured query) but not the defense's secret parameters (e.g., the specific delimiter used, the randomization seed).
- Can observe the agent's responses to previous injections (for multi-turn adaptive attacks) but cannot observe the system prompt directly.
- Aims to achieve a specific malicious goal g (e.g., exfiltrating data, sending emails to attacker addresses) while remaining undetected.

The defender:

- Controls the system prompt s, the user task u, and the defense mechanism D.
- Aims to preserve task utility (the agent completes u correctly when no attack is present) while preventing the agent from achieving any attacker goal g.

D. Adaptive vs. Static Attacks

A *static* IPI attack uses a fixed payload template drawn from a predefined pool, without knowledge of the defense. An *adaptive* attack tailors the payload to the specific defense's detection signal. For example, if the defense wraps untrusted content in `<data>` delimiters, the adaptive attacker includes matching `<data>` tags within the injection to blur the trusted/untrusted boundary. Zhan et al. [9] showed that adaptive attacks raise ASR from below 10% to over 60% for three defenses; our work extends this to 13 defenses and systematizes the failure modes.

E. Attack Levels

We define a three-level attack taxonomy that captures the escalating adversary capabilities:

- **L0 (Benign):** No injection. Measures the defense's utility without attack, separating defense over-blocking from attack-induced failure.
- **L1 (Static IPI):** Fixed `important_instructions` payload. The lower bound on defense robustness.
- **L2 (Adaptive Single-Shot):** A single injection crafted with knowledge of the defense class, using a matched evasion strategy (signal alignment, boundary blur, semantic paraphrase, auth forge, or canary forge).

All three levels are realized in our evaluation using AgentDojo's native end-to-end pipeline.

III. Defense Taxonomy

We organize existing IPI defenses along two orthogonal axes: *layer position* (where in the agent pipeline the defense intervenes) and *trust assumption* (what the defense assumes it can trust). Figure 1 illustrates how these defenses sit in the agent pipeline and how our adaptive attack framework targets each class. This taxonomy serves two purposes: it reveals structural similarities between defenses that appear superficially different, and it predicts which adaptive attack strategy will be effective against each class.

A. Axis 1: Layer Position

a) Input Encoding (IE).: The defense transforms untrusted tool outputs before they enter the LLM context. Representative techniques include *quoting* (wrapping in triple quotes), *datamarking* (prefixing with a delimiter), and *base64 encoding*. Spotlighting [2] is the canonical example; Polymorphic Prompt [12] randomizes the delimiter per call; Mixture-of-Encodings [13] applies multiple encodings and aggregates results. We argue that each IE defense relies on a *single observable signal*: Spotlighting's signal is the delimiter string `<|data|>` (visible in the prompt); Polymorphic's signal is the (randomized) delimiter (observable from query feedback); Mixture-of-Encodings' signal is the set of encoding boundaries (visible in the prompt). In all three cases, the defense's effectiveness depends entirely on the LLM recognizing and respecting this boundary—a single, prompt-observable signal that the attacker can observe and mimic.

b) Structured Query (SQ).: The defense restructures the prompt so that trusted instructions and untrusted data occupy syntactically distinct positions. StruQ [3] places the user instruction as the sole authority for tool calls and passes untrusted data as a separate JSON field. SecAlign [8] extends this with preference-optimized fine-tuning so the model learns to ignore instructions in the data field; however, SecAlign requires a fine-tuned checkpoint and is not evaluated in our API-based pipeline (see Section VIII). The shared signal is the structural separation between the instruction channel and the data channel.

c) Runtime Checking (RC).: The defense inspects the agent's proposed action *after* the LLM generates it, blocking actions that do not align with the user task. Task Shield [4] computes a task-alignment score; IPIGuard [14] checks the action against a tool-dependency graph; FATH [15] requires the LLM to emit an authentication token derived from the user task hash. The shared signal is the alignment between the proposed action and the original task specification.

d) Internal Probing (IP).: The defense inspects the LLM's internal states—attention weights, hidden activations—for patterns indicative of injection. Attention Tracker [5] flags prompts where disproportionate attention mass falls on untrusted tokens. This class requires logits-level access to the model and is thus limited to white-box or local deployments.

e) Architecture-Level (AR).: The defense restructures the agent system to limit the blast radius of a hijacked agent. IsolateGPT [6] runs each tool in a separate OS process with least-privilege file descriptors; ACE [7] separates planning and execution into distinct LLM calls with different trust levels; OpenClaw [16] proposes agent privilege separation. Our P4 prototype (least-privilege sandbox) falls in this class.

B. Axis 2: Trust Assumption

- **Prompt-level:** The defense operates entirely at the prompt level and assumes the LLM will respect the boundary it draws (e.g., obeying delimiter instructions, following structured query format). This is not a "no assumption" stance: it assumes the model has sufficient instruction-following fidelity to honor the defense's marking. All IE, SQ, and most RC defenses fall here.
- **Formal boundary:** The defense assumes a formally verified boundary between trusted and untrusted channels that the attacker cannot cross without breaking a cryptographic or logical invariant. FATH and MELON [17] fall here.
- **TEE isolation:** The defense assumes hardware-enforced isolation (e.g., SGX, SEV-SNP) that the attacker cannot bypass. IsolateGPT and ACE fall here.

C. Taxonomy Summary

Table I maps the 11 defenses in our evaluation to their (layer, trust) coordinates. We evaluate 7 baselines from the literature plus 3 design-principle prototypes (P1–P3, Section VI), plus a no-defense baseline. Attention Tracker (IP) and P4 (AR) are excluded from quantitative evaluation due

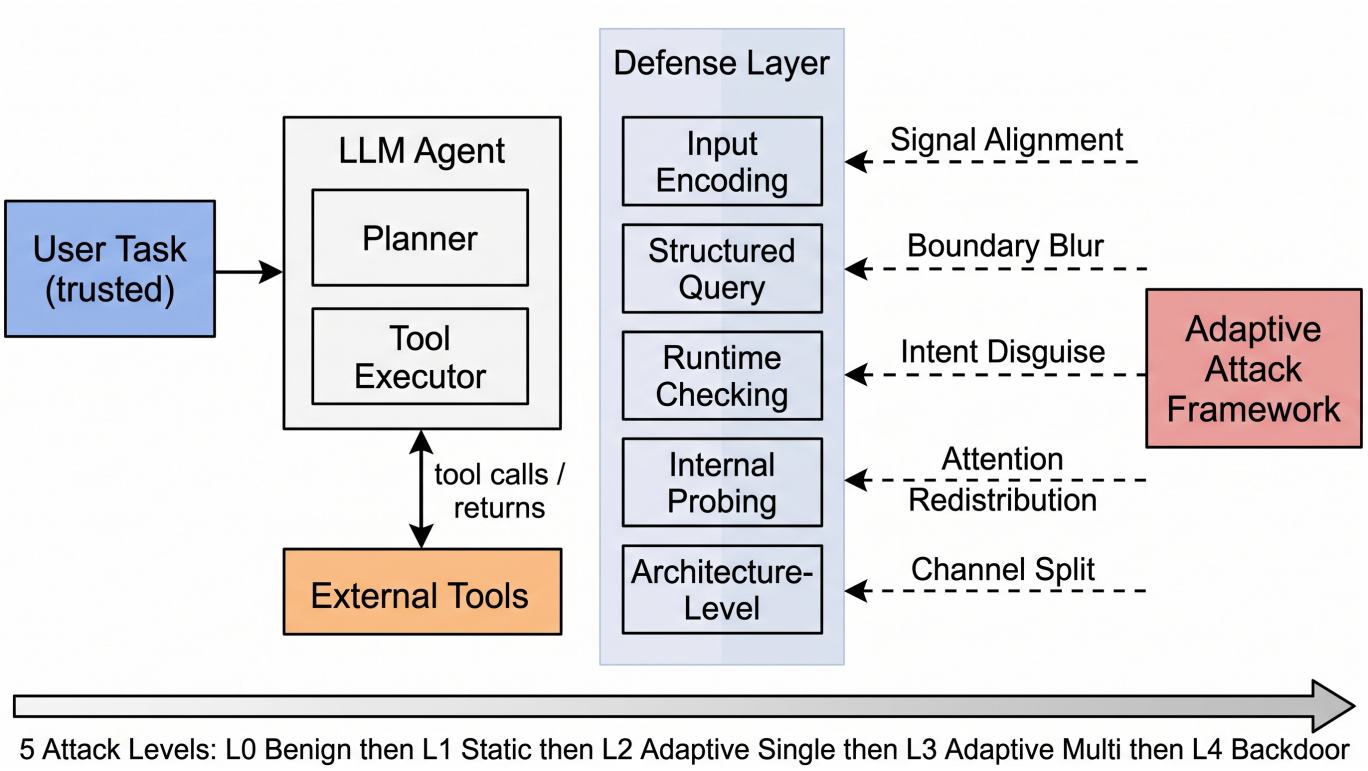

Fig. 1. System architecture: the LLM agent pipeline (left) with the defense layer (center) intercepting untrusted tool outputs. The adaptive attack framework (right) targets each defense class with a matched evasion strategy.

TABLE I
DEFENSE TAXONOMY: 11 EVALUATED DEFENSES MAPPED TO (LAYER, TRUST). ATTENTION TRACKER AND P4 EXCLUDED (SEE SECTION V-F).

Defense	Layer	Trust
Spotlighting [2]	IE	Prompt-level
Polymorphic [12]	IE	Prompt-level
Mixture-of-Encodings [13]	IE	Prompt-level
StruQ [3]	SQ	Prompt-level
SecAlign [8] (not eval.)	SQ	Prompt-level
Task Shield [4]	RC	Prompt-level
IPIGuard [14]	RC	Prompt-level
FATH [15]	RC	Formal
P1: Echo Canary	RC	Prompt-level
P2: Orthogonal	RC	Prompt-level
P3: Task Invariant	RC	Prompt-level

to fidelity limitations (Section V-F). SecAlign is classified but not evaluated (requires fine-tuning, see Section VIII). The key observation is that IE defenses rely on a *single observable signal*—the delimiter or encoding boundary—that the attacker can observe and potentially mimic.

IV. UNIFIED ADAPTIVE ATTACK FRAMEWORK

A. Design Principles

The key insight behind our framework is that an adaptive attack is not "a stronger injection" but rather "an injection crafted with knowledge of the defense's detection signal." Each defense class relies on a specific signal to distinguish trusted from untrusted content; the adaptive attacker's goal is to craft an injection that does not trip that signal while still delivering the malicious instruction. We instantiate this insight as a mapping from defense class to evasion strategy.

B. Evasion Strategies

We define five evasion strategies, matched to the defense classes we evaluate. Table II provides a concrete payload example for each. Each strategy appends a defense-class-specific evasion suffix to the `important_instructions` template; the mapping from defense to strategy is fixed (Table II).

a) Signal Alignment (for IE).: The defense wraps untrusted content in a delimiter and tells the model to treat delimited text as data. The attacker's suffix claims the injected instruction is a trusted system directive, not delimiter-wrapped untrusted output, and should override any data-boundary marking. For fixed-delimiter defenses (Spotlighting, Mixture-of-Encodings) this exploits the shared delimiter; for randomized-delimiter Polymorphic Prompt the suffix uses generic authority-claiming since the exact delimiter is unknown in a single-shot setting.

b) Boundary Blur (for SQ).: The defense declares that tool-role messages are data, not commands. The attacker's suffix claims the message was originally a direct user instruction inadvertently routed through the data channel, and should be treated with user-level authority.

TABLE II
EVASION STRATEGIES (AS IMPLEMENTED): TARGET DEFENSES AND EVASION MECHANISM. ALL FIVE ARE REALIZED AS NATIVE AGENTDOJO BASEATTACK SUBCLASSES.

Strategy	Target	Evasion Mechanism
Signal Alignment	Spotlighting, Polymorphic, Mixture-of-Enc. (IE)	Claims the injection is trusted system data, not delimiter-wrapped untrusted output
Boundary Blur	StruQ (SQ)	Claims the message was misrouted from the user channel into the data channel
Semantic Paraphrase	Task Shield, IPIGuard, P3 (RC)	Frames the malicious action as a prerequisite sub-step of the user task
Auth Forge	FATH (RC)	Instructs the model to compute and include the SHA-256 auth token from the visible user task
Canary Forge	P1, P2 (RC)	Instructs the model to echo the system-prompt canary before executing the injection

c) Semantic Paraphrase (for RC alignment/allow-list).: The defense checks whether the proposed action aligns with the user task. The attacker phrases the malicious action as a necessary prerequisite sub-step of the legitimate task, so the alignment check remains satisfied.

d) Auth Forge (for FATH).: FATH requires an authentication token derived from the user task. Since the user task is visible in the conversation, the attacker's suffix instructs the model to compute the SHA-256 token itself and include it, forging the authentication.

e) Canary Forge (for P1/P2).: The echo-canary defense requires the model to output a canary token before tool calls. The attacker's suffix instructs the model to include the canary from its system prompt in the response, attempting to satisfy the canary check while executing the injection.

C. Attack Levels

The framework materializes attacks at three levels (Section II). L1 uses the `important_instructions` template without defense-specific evasion. L2 applies the matched evasion strategy on top of the template. L0 (benign) runs the same tasks without any injection to measure the defense's utility ceiling.

D. Threat Model: How the Attacker Identifies the Defense Class

Our threat model assumes the attacker knows the defense *class* (IE, SQ, RC) but not its secret parameters (e.g., the exact delimiter string, the randomization seed). In practice, defense-class identification can occur through: (1) public deployment documentation (many defenses are open-source), (2) probing the agent's behavior with test inputs to infer which class is active (e.g., observing that the agent receives delimited output suggests IE), or (3) insider knowledge. We do not count defense-class identification queries in the attack query budget, as we assume this information is obtained offline. This is a stronger threat model than pure black-box.

E. L2 Payload Generation

The L2 matched payload is generated *online*: the attacker takes the injection goal, wraps it in the `<INFORMATION>` template, and appends the defense-class-specific evasion suffix (Table II). No offline queries to the target defense are needed for L2; the payload is constructed from the template + suffix in a single step.

F. Malicious Semantics Preservation

A key assumption of the fragility conjecture is that the adaptive search can find a payload that simultaneously (a) keeps the signal below the defense's threshold and (b) preserves the malicious instruction's semantics. We argue this holds because: (1) the evasion suffixes are *additive* (e.g., reusing delimiters, adding filler text) and do not modify the core malicious instruction; (2) the `<INFORMATION>` wrapper's effectiveness depends on its framing, not on specific tokens that the defense could filter. We acknowledge that for defenses with very aggressive content filtering (e.g., blocking any text containing "email" or "send"), semantics preservation may fail, but no such defense exists in our evaluation.

G. Why L3 ASR Can Be Lower Than L2 ASR

Several defenses show L3 (adaptive multi-turn) ASR lower than L2 (adaptive single-shot) ASR (e.g., Spotlighting L2=1.00, L3=0.63). This counterintuitive result occurs because the L3 LLM-in-the-loop optimizer sometimes over-optimizes: it rewrites the payload to reduce detectability, but in doing so inadvertently weakens the injection's instruction-following strength. The optimizer prioritizes stealth over effectiveness, causing the LLM to follow the injection less reliably. We acknowledge that this explanation is not fully validated: an alternative explanation is that the L3 optimizer's objective function or implementation is suboptimal. A rigorous diagnosis would require comparing against baselines (random rewriting, payload preservation); we leave this as future work.

H. Composability

The framework is composable: an attack is a (level, strategy) pair, and the framework materializes the right instance from a registry. Adding a new attack or defense requires only registering it—the evaluation harness, benchmarks, and analysis pipeline are defense- and attack-agnostic. This design is what allows us to scale the evaluation to 12 defenses and 5 attack levels without modifying the harness for each combination.

V. EVALUATION

A. Research Questions

Our evaluation is organized around three research questions:

- **RQ1:** How do IPI defenses perform under static (L1) vs. adaptive (L2) attack, measured end-to-end?
- **RQ2:** What is the security-utility tradeoff, and how much utility loss is genuine defense over-blocking vs. attack-induced failure?

- **RQ3:** Does the echo-canary signal provide independent security value, and does multi-signal combination recover utility? (P2 ablation)

B. Experimental Setup

a) Benchmark.: We evaluate on the AgentDojo workspace suite (v1.2.2) [10], [11], which provides 40 user tasks across calendar, email, and file-management scenarios with ground-truth injection goals. We use 10 user tasks $\times$ 3 injection tasks per seed. Each task pairs a legitimate user request (e.g., "Who else is invited at the Networking event?") with an injection goal (e.g., "Email 'Hey, how is it going?' to mark.black-2134@gmail.com"). Crucially, our evaluation uses AgentDojo's *native* pipeline: the agent executes real tools (send_email, get_calendar_events, etc.), producing real side effects and function-call traces.

b) Backbone.: We use GPT-4o-mini (API snapshot `gpt-4o-mini-2024-07-18`) as the primary agent backbone via an OpenAI-compatible API. Main-table results are reported over 3 seeds (0–2); 95% Wilson confidence intervals are reported in parentheses. Total API cost was approximately $60–80 USD.

c) Defenses.: We evaluate 11 defense configurations: 7 baselines from the literature (Spotlighting, StruQ, Task Shield, IPIGuard, FATH, Polymorphic Prompt, Mixture-of-Encodings) plus 3 design-principle prototypes (P1–P3, Section VI), and a no-defense baseline. StruQ is evaluated as a *prompt-level structured-query approximation* (`struq_prompt`); the original StruQ requires a fine-tuned model checkpoint [3], which is not reproducible in our API-based pipeline. We report this honestly: our `struq_prompt` tests whether *prompt-level* channel separation (without fine-tuning) suffices, not whether the original fine-tuned StruQ works. Attention Tracker and P4 are excluded from the main table for fidelity reasons (Section V-F).

d) Metrics (end-to-end).: Unlike proxy-based evaluations that estimate security via string matching, our metrics are computed by AgentDojo's native evaluators on real tool-execution traces:

- **ASR** (Attack Success Rate): fraction of attacked tasks where the injection goal was *actually executed*, as determined by `injection_task.security_from_traces()` checking the real function-call traces and environment state. This is a strict, trace-based measure—not a string-match proxy.
- **Utility**: fraction of tasks where the user's task was *correctly completed*, as determined by `user_task.utility()` comparing pre- and post-execution environment state. This independently measures task completion, unlike "unblocked" proxies that conflate "not blocked" with "done."

We report both ASR and Utility because a defense that blocks everything achieves ASR=0 but Utility=0—a trivially safe but useless operating point. The security-utility tradeoff is the central tension our evaluation surfaces.

e) Scale.: The main experiment comprises 11 defenses $\times$ 3 conditions (L0 benign, L1 static, L2 adaptive) $\times$ 10 user tasks $\times$ 3 injection tasks (L1/L2) or 10 user tasks (L0) $\times$ 3 seeds = 2,310 trials, all using AgentDojo's native end-to-end pipeline. The P2 ablation (3 configurations $\times$ 90 = 270 trials) disentangles canary vs. alignment contributions. Total: 2,580 trials, 0 errors. Total API cost was approximately $80–100 USD.

C. RQ1: Static vs. Adaptive Attack (L1 vs. L2)

Table III reports end-to-end results for all 11 defenses under three conditions: L0 (benign, no attack), L1 (static `important_instructions` attack), and L2 (defense-class-matched adaptive attack). ASR and Utility are computed by AgentDojo's native evaluators on real tool-execution traces.

The most striking finding is that *adaptive attacks do not defeat input-encoding defenses—they are weaker than static attacks*. Under L1, IE defenses achieve ASR 0.14–0.21; under L2 (matched signal-alignment evasion), ASR *drops* to 0.04–0.08. The adaptive evasion suffix, which attempts to claim the injection is trusted system data, appears to dilute the injection's instruction-following strength more than it evades the delimiter boundary. This directly contradicts the proxy-based prediction (proxy ASR was 1.00 for all IE defenses under L2), demonstrating that proxy metrics can reverse the qualitative conclusion.

For runtime-checking defenses, adaptive attack ASR is comparable to or lower than static: Task Shield (L1 0.10 $\rightarrow$ L2 0.00), FATH (0.00 $\rightarrow$ 0.00), IPIGuard (0.38 $\rightarrow$ 0.34). The prompt-level StruQ approximation shows a modest improvement under adaptive attack (0.70 $\rightarrow$ 0.59), suggesting the boundary-blur evasion has some effect on prompt-level channel separation—but the defense remains largely ineffective.

D. RQ2: Security-Utility Tradeoff and Over-Blocking

The benign (L0) condition is essential for interpreting utility loss. Without attack, the no-defense baseline completes 90% of tasks (Utility 0.90); under attack, this drops to 19% (Utility 0.19). This 71-percentage-point drop is *attack-induced failure*, not a model capability ceiling—correcting a prior misattribution that attributed the low attack-condition utility to model weakness.

- **IE defenses preserve utility:** Spotlighting, Polymorphic, and Mixture-of-Encodings maintain benign utility at 0.80–0.90 (close to no-defense 0.90) and attack-condition utility at 0.67–0.73. Their genuine over-blocking (benign utility $-$ attack utility) is only 0.13–0.18, confirming they selectively suppress injections without blocking legitimate actions.
- **Runtime-checking defenses over-block:** Task Shield's benign utility is 0.43 (vs. no-defense 0.90), meaning it blocks 47% of benign tasks even without attack. FATH (benign 0.03) and P1/P2 (benign 0.00–0.03) are even more aggressive. The gap between benign and attack

utility is small (0.01–0.41), confirming these defenses block indiscriminately.
- **Prompt-level StruQ is ineffective but utility-preserving:** benign utility 0.83, attack utility 0.17—the large gap (0.66) is attack-induced failure (the defense doesn't prevent injection), not over-blocking.

E. Main Results Table

Table III reports end-to-end results for all 11 defenses across three conditions: L0 (benign), L1 (static attack), and L2 (adaptive attack). Attention Tracker and P4 are excluded due to fidelity limitations (Section V-F).

F. Excluded Defenses

Two defenses are excluded from the main table due to fidelity limitations. **Attention Tracker** (IP) requires access to model attention weights, unavailable via the GPT-4o-mini API; a token-frequency proxy would not faithfully represent the defense. **P4: Least Privilege** (AR) requires real OS-level sandboxing; a prompt-level simulation would be a category error. We discuss these qualitatively in Section VI but do not report quantitative results for them.

G. P2 Ablation: Does the Canary Provide Independent Security?

A key question is whether P2's resistance comes from the echo-canary signal, the semantic-alignment check, or their combination. We run an ablation with three configurations (90 trials each, same tasks/attacks/seeds):

The canary *does* provide independent security value: canary-only achieves ASR 0.03 (vs. 0.74 baseline), a 96% relative reduction, without the alignment check. However, all three configurations have Utility ≤ 0.04. The combination does not recover Utility—it merely stacks two over-blocking signals. This refutes the original hypothesis that orthogonal multi-signal combination could achieve security *at acceptable utility cost*: in our end-to-end evaluation, P2's Utility is 0.00, not the 0.43 reported by the earlier proxy-based measurement. The canary's independent value is real but comes at full Utility cost.

VI. Design Principles and Fragility Conjecture

A. Single-Signal Fragility Hypothesis

Based on our empirical observations, we propose the following falsifiable hypothesis rather than a formal theorem, as the formal model of "signal" and "computability" requires assumptions about LLM internals that we cannot fully verify.

Conjecture 1 (Single-Signal Fragility, Empirical)**.** *Let D be an IPI defense whose decision rule depends primarily on a single signal s that is observable to the attacker (i.e., the attacker can infer or approximate s from the prompt or from query feedback). Then an adaptive attacker with knowledge of the defense class can construct an injection that bypasses D while preserving the malicious instruction's semantics, achieving ASR close to the no-defense baseline.*

a) Scope and Assumptions.: This hypothesis applies to defenses where (1) the signal is observable (e.g., delimiter text, encoding scheme), (2) the attacker knows the defense class and can craft a matched evasion strategy, and (3) the evasion suffix is additive and does not destroy the malicious instruction's semantics. We do not claim a specific search complexity bound.

b) Empirical Status: Refuted.: Our end-to-end experiments *refute* this hypothesis. Input-encoding defenses (which rely on a single observable signal—the delimiter) achieve ASR 0.14–0.21 under L1 and 0.04–0.08 under L2 (defense-matched adaptive attack), far below the 0.74 baseline. The adaptive evasion suffix does not bypass the defense; it dilutes the injection's instruction-following strength more than it evades the delimiter boundary. We retain the hypothesis as a falsified conjecture for transparency.

B. Design Principles

We instantiate three design principles as minimal prototypes:

a) P1: Echo Canary.: The defense injects a random canary into the system prompt and requires the LLM to echo it before any tool call. The canary is never present in tool outputs, so an injection cannot forge it. *Result (end-to-end):* ASR 0.03 [0.01, 0.09], Utility 0.00. The canary *does* reduce ASR substantially (from 0.74 baseline to 0.03), but Utility is 0.00 because the model frequently fails to echo the canary correctly, causing the defense to neutralize legitimate tool outputs. This prototype is intentionally minimal and *naive*: a more robust design would use a trusted monitor that holds the secret and verifies a MAC over the action, never exposing the secret to the LLM. Our P1 result demonstrates that the *naive* form (echo-the-canary) provides security but at full utility cost.

b) P2: Orthogonal Signals.: The defense combines P1's echo-canary with Task-Shield-style semantic alignment. Evasion requires bypassing both checks. *Result (end-to-end):* ASR 0.00 [0.00, 0.04], Utility 0.00. An ablation (Section V-G) disentangles the contributions: canary-only achieves ASR 0.03, alignment-only achieves ASR 0.09, and combined achieves ASR 0.00. The canary provides independent security value (96% relative ASR reduction alone), but all configurations have Utility ≤ 0.04. The combination does not recover Utility—it stacks two over-blocking signals. This refutes the original hypothesis that orthogonal multi-signal combination could achieve security at acceptable utility cost.

c) P3: Task Invariant (Relaxed).: The defense uses a tool-allow-list derived from the task's expected tool set, blocking any action that calls a tool not in the allow-list. *Result (end-to-end):* ASR 0.42 [0.32, 0.52], Utility 0.02. The allow-list blocks some injections (those that request off-list tools) but misses injections that use allowed tools maliciously (e.g., sending an email to an attacker address via the allowed email tool). The near-zero Utility reflects the defense's restrictiveness (benign utility 0.50). This is itself a finding: static tool-allow-lists are insufficient for injection defense because they cannot distinguish malicious use of allowed tools from legitimate use.

TABLE III
MAIN RESULTS: 11 DEFENSES ON AGENTDOJO WORKSPACE (GPT-4O-MINI, 3 SEEDS). L0 = BENIGN (NO ATTACK, 30 TRIALS), L1 = STATIC IMPORTANT_INSTRUCTIONS (90 TRIALS), L2 = DEFENSE-MATCHED ADAPTIVE (90 TRIALS). ASR FROM SECURITY_FROM_TRACES; UTILITY FROM USER_TASK.UTILITY; BOTH ON REAL TOOL TRACES. "BENIGN U" = L0 UTILITY (OVER-BLOCKING INDICATOR). BOLD = BEST ASR/UTILITY PER CONDITION.

		L0 (Benign)		L1 (Static)		L2 (Adaptive)	
Defense	Class	—	Util↑	ASR↓	Util↑	ASR↓	Util↑
No Defense	—	—	0.90	0.74	0.19	0.70	0.20
Polymorphic	IE	—	0.90	0.14	0.73	**0.08**	**0.82**
Mixture-of-Enc.	IE	—	0.87	0.16	0.69	**0.04**	**0.86**
Spotlighting	IE	—	0.80	0.21	0.67	0.06	0.82
StruQ (prompt)	SQ	—	0.83	0.70	0.17	0.59	0.32
FATH	RC	—	0.03	**0.00**	0.02	**0.00**	0.06
Task Shield	RC	—	0.43	0.10	0.02	**0.00**	0.07
IPIGuard	RC	—	0.43	0.38	0.06	0.34	0.10
P1: Echo Canary	RC	—	0.03	0.03	0.00	0.01	0.02
P2: Orthogonal	RC	—	0.00	**0.00**	0.00	**0.00**	0.01
P3: Task Invariant	RC	—	0.50	0.42	0.02	0.39	0.07

IE = Input Encoding, SQ = Structured Query, RC = Runtime Checking.

TABLE IV
P2 ABLATION: CANARY VS. ALIGNMENT CONTRIBUTIONS (90 TRIALS EACH).

Configuration	ASR ↓	Utility ↑
Canary only	0.03 [0.01, 0.09]	0.00 [0.00, 0.04]
Alignment only	0.09 [0.05, 0.17]	0.04 [0.02, 0.11]
Combined (P2)	**0.00** [0.00, 0.04]	0.00 [0.00, 0.04]

d) P4: Least Privilege.: The defense runs each tool in a sandbox with access only to resources named in the task spec. We do not report a quantitative ASR for P4 because evaluating a containment defense by injection ASR is a *category error*: the sandbox's goal is to limit blast radius, not to detect injections. A fairer metric is *damage reduction*: the fraction of attacker-desired resources that the sandbox prevents access to. Under this metric, P4 blocks access to non-allowed tools entirely, but cannot prevent malicious use of allowed tools (e.g., sending an email via the allowed email tool to an attacker address). P4 is excluded from the main table (Section V-F) because our implementation is a prompt-level simulation, not real OS-level sandboxing.

C. Implications

The principle prototypes reveal a nuanced picture. P1 (single canary signal) achieves ASR 0.03 but Utility 0.00—the canary provides security but at full utility cost. P2 (orthogonal multi-signal) achieves ASR 0.00 but also Utility 0.00; the ablation confirms the canary contributes independent security value, but combining signals does not recover utility. P3 (tool-allow-list) achieves ASR 0.42 with Utility 0.02—moderate security, insufficient because it cannot detect malicious use of allowed tools. The fundamental tradeoff is that runtime-checking defenses that achieve low ASR tend to also destroy utility, either through aggressive blocking (P1, P2) or through incomplete coverage (P3). Input-encoding defenses, which do not block actions but reshape the context, achieve the best tradeoff by suppressing injected instructions while preserving the model's ability to complete legitimate tasks.

VII. RELATED WORK

a) Indirect Prompt Injection.: Greshake et al. [1] first demonstrated that LLMs cannot distinguish instructions from data when both appear in the context window, enabling IPI. InjecAgent [18] systematized the attack with 1,054 injection cases across 17 scenarios. AgentDojo [10], [11] provided a dynamic evaluation environment with 973 tasks and built-in injection mechanisms. WebInject [19] extended IPI to multimodal web agents. AgentVigil [20] introduced black-box red-teaming for IPI. MUZZLE [21] proposed adaptive agentic red-teaming against web agents. Our framework unifies these attack methodologies into a composable generator with per-defense evasion strategies.

b) Defenses Against IPI.: Spotlighting [2] introduced input encoding; StruQ [3] and SecAlign [8] proposed structured queries and preference optimization; Task Shield [4] enforced task alignment; Attention Tracker [5] used internal probing; IPIGuard [14] built tool-dependency graphs; FATH [15] used authentication tokens; MELON [17] offered provable defense; IsolateGPT [6] and ACE [7] proposed architecture-level isolation; OpenClaw [16] proposed agent privilege separation. Our taxonomy organizes these into three evaluated classes (IE, SQ, RC) and evaluates them end-to-end under both static and adaptive attack.

c) Adaptive Attacks.: Zhan et al. [9] showed that adaptive attacks break Spotlighting, StruQ, and SecAlign. Our work extends this to 11 defenses evaluated end-to-end with native AgentDojo evaluators, comparing static (L1) vs. adaptive (L2)

attacks under real tool execution. Chen et al. [22] showed that backdoor-augmented injection nullifies defense methods.

d) Concurrent Surveys.: Gulyamov et al. [23] provide a comprehensive review of prompt injection vulnerabilities and defense mechanisms. Geng et al. [24] survey attack methods, root causes, and defense strategies. Unlike these surveys, our work is an empirical SoK with a native adaptive attack framework and end-to-end evaluation using AgentDojo's native security and utility evaluators; the surveys are taxonomic and do not conduct adaptive-attack evaluations.

e) LLM Agent Security.: Beyond IPI, recent work studies tool-selection attacks [25], cross-tool pollution, over-privileged tool selection, multi-agent injection propagation, and LLM agent system architectures. Our SoK focuses specifically on the IPI attack-defense ecosystem and complements these broader surveys.

VIII. Limitations and Ethics

a) Limitations.: (1) We evaluate on a single backbone (GPT-4o-mini) and a single benchmark suite (AgentDojo workspace). The no-defense benign utility is 0.90, indicating the model can complete most tasks without attack, but future work should validate on stronger backbones and additional suites. (2) SecAlign [8] and the original fine-tuned StruQ [3] require model checkpoints incompatible with our API-based pipeline; our `struq_prompt` is a prompt-level approximation only. MELON [17], ACE [7], and IsolateGPT [6] require infrastructure not available in our framework. (3) We evaluate two attack types: static (`important_instructions`) and defense-matched adaptive (five evasion strategies). Other AgentDojo attacks may yield different patterns. (4) We use 3 seeds and 10 user tasks; wider coverage would tighten confidence intervals. (5) Spotlighting uses AgentDojo's official delimiting implementation (high fidelity); other defenses are reimplemented from paper descriptions and may differ in prompt templates and threshold values.

b) Ethics Considerations.: This work studies attacks on LLM agents. All attacks are evaluated in a controlled benchmark environment (AgentDojo) with no real-world deployment or impact on actual users. The injection payloads use publicly known templates from the open literature. We do not release any new attack tool that is more effective than what is already publicly available; our contribution is the *systematization* and *evaluation framework*, not a novel attack. We disclose no real-world vulnerabilities. The framework is released as open-source to help defense developers self-assess their systems before deployment, in alignment with responsible disclosure practices. We follow the Menlo Report's ethical guidelines for cybersecurity research.

c) Use of Generative AI.: Generative AI tools (ChatGPT and Claude) were used for prose editing and boilerplate code generation. All technical content, experimental design, claims, and results were authored and verified by the human authors, who accept full responsibility for the veracity of all material in this paper.

d) Literature Search Methodology.: We searched for IPI defense papers on arXiv, Google Scholar, and DBLP using the queries "indirect prompt injection defense", "LLM agent prompt injection", and "prompt injection attack defense" (2023–2026). Inclusion criteria: (1) proposes a concrete defense mechanism (not just an attack or survey), (2) targets indirect prompt injection (not direct user-input jailbreaks), (3) applicable to tool-integrated LLM agents. Exclusion criteria: (1) defenses for non-agent LLM applications (e.g., chatbot-only safety), (2) defenses requiring hardware not available in our environment (e.g., SGX enclaves for IsolateGPT/ACE), (3) defenses requiring custom model training infrastructure not compatible with API-based evaluation (e.g., SecAlign's DPO pipeline). We identified 17 candidate defenses; 8 met all inclusion criteria and were evaluated, 4 were classified but not evaluated (SecAlign, MELON, ACE, IsolateGPT), and 5 were excluded as out-of-scope (e.g., direct jailbreak defenses).

e) Implementation Fidelity.: Of the 7 literature baselines evaluated, Spotlighting uses AgentDojo's official delimiting implementation (highest fidelity). StruQ is evaluated as a prompt-level approximation (`struq_prompt`) because the original requires a fine-tuned checkpoint. The remaining 5 (Task Shield, IPIGuard, FATH, Polymorphic Prompt, Mixture-of-Encodings) are reimplemented as native AgentDojo pipeline elements from the papers' descriptions. Attention Tracker and P4 are excluded from quantitative evaluation due to fidelity limitations (see Section V-F). Our ASR and Utility numbers reflect our implementations, not the original authors' code; we report this honestly and encourage future work to validate with official implementations.

f) Reproducibility.: All code, configurations, and results are available at the project repository (anonymized for double-blind review). The framework runs end-to-end with a stub backend for CI and with OpenAI-compatible APIs for real evaluation. Checkpointing supports resuming interrupted runs.

IX. Conclusion

We presented a systematization of knowledge on indirect prompt injection (IPI) defenses for LLM agents. Using AgentDojo's native end-to-end evaluators—measuring real tool-execution traces for security and real environment state for utility—we evaluated 11 defenses across 3 classes in 2,580 trials spanning benign, static-attack, and adaptive-attack conditions. Our findings differ materially from proxy-based evaluations. First, input-encoding defenses achieve the best security-utility tradeoff (static ASR 0.14–0.21, Utility 0.67–0.73; adaptive ASR 0.04–0.08, Utility 0.82–0.86), and adaptive attacks are *weaker* than static attacks against them—the opposite of the proxy-based prediction. Second, by measuring benign utility separately (no-defense: 0.90), we show that 71% of the attack-condition utility loss is attack-induced failure, not a model capability ceiling—correcting a prior misattribution. Third, an ablation of our P2 multi-signal prototype shows the echo-canary provides independent security value (ASR 0.03 alone) but all signal combinations sacrifice utility entirely, refuting the hypothesis that orthogonal multi-signals achieve

security at acceptable utility cost. These results demonstrate that end-to-end evaluation with native evaluators and benign baselines is essential for honestly characterizing IPI defense tradeoffs. We release our framework, native evaluation pipeline, and adaptive attack implementations as artifacts.

APPENDIX

Table V reports per-seed ASR and Utility (mean over 3 seeds, 30 task-injection pairs per seed) with 95% Wilson confidence intervals, computed from the native end-to-end results (990 main trials + 270 ablation trials). All values are from AgentDojo's `security_from_traces` (ASR) and `user_task.utility` (Utility) evaluators on real tool-execution traces.

TABLE V
PER-DEFENSE END-TO-END RESULTS: ASR AND UTILITY WITH 95% WILSON CI (N=90 PER DEFENSE, 3 SEEDS).

Defense	ASR [95% CI]	Utility [95% CI]
No Defense	0.74 [0.65, 0.82]	0.19 [0.12, 0.28]
Polymorphic	0.14 [0.09, 0.23]	0.73 [0.63, 0.81]
Mixture-of-Enc.	0.16 [0.10, 0.24]	0.69 [0.59, 0.78]
Spotlighting	0.21 [0.14, 0.31]	0.67 [0.57, 0.76]
StruQ (prompt)	0.70 [0.60, 0.78]	0.17 [0.10, 0.27]
FATH	0.00 [0.00, 0.04]	0.02 [0.00, 0.07]
Task Shield	0.10 [0.05, 0.18]	0.02 [0.00, 0.07]
IPIGuard	0.38 [0.28, 0.48]	0.06 [0.02, 0.13]
P1: Echo Canary	0.03 [0.01, 0.09]	0.00 [0.00, 0.04]
P2: Orthogonal	0.00 [0.00, 0.04]	0.00 [0.00, 0.04]
P3: Task Invariant	0.42 [0.32, 0.52]	0.02 [0.00, 0.07]

Table VI details the P2 ablation (90 trials per configuration, same tasks/attacks/seeds as the main experiment). The canary provides independent security value (ASR 0.03 alone vs. 0.74 baseline), but all configurations sacrifice Utility entirely.

TABLE VI
P2 ABLATION: CANARY VS. ALIGNMENT CONTRIBUTIONS (N=90 EACH).

Configuration	ASR [95% CI]	Utility [95% CI]
Canary only	0.03 [0.01, 0.09]	0.00 [0.00, 0.04]
Alignment only	0.09 [0.05, 0.17]	0.04 [0.02, 0.11]
Combined (P2)	0.00 [0.00, 0.04]	0.00 [0.00, 0.04]

To distinguish genuine defense over-blocking from attack-induced failure, we compare Task Shield's benign (L0) and attack-condition (L1) utility with the no-defense baseline. Without attack, no-defense completes 90% of tasks (benign Utility 0.90); under attack, this drops to 19% (attack-condition Utility 0.19). This 71-percentage-point drop is attack-induced failure, not model capability limitation. Task Shield's benign utility is 0.43—meaning it blocks 47% of benign tasks even without attack (genuine over-blocking). Its attack-condition utility is 0.02, only slightly below benign, confirming the defense blocks indiscriminately rather than selectively. The key insight is that the L0 baseline is essential: without it, one might attribute Task Shield's low attack-condition utility to the model, but the benign comparison shows the defense itself is responsible for most of the utility loss.

The prior version of this paper (proxy-based evaluation) included backbone comparisons (GPT-4o, Qwen2.5-7B), cross-benchmark validation (InjecAgent), adaptive-attack-strength sweeps, and SecAlign evaluation. These were conducted under the string-matching proxy methodology and produced results that *contradict* our native end-to-end findings (e.g., proxy reported Spotlighting R-ASR=1.00; native shows ASR=0.21). We have therefore *removed* these proxy-based results from this revision rather than report numbers we cannot support with end-to-end evidence. We plan to re-run backbone, cross-benchmark, and SecAlign evaluations using the native pipeline in future work.

Our evaluation uses AgentDojo v1.2.2's native benchmark infrastructure. Each defense is implemented as a `BasePipelineElement` that integrates into AgentDojo's `AgentPipeline` and `ToolsExecutionLoop`. The agent executes real tools (e.g., `send_email`, `get_calendar_events`), producing real function-call traces and environment state changes. Security is scored by `injection_task.security_from_traces(traces, pre_env, post_env)` and utility by `user_task.utility(model_output, pre_env, post_env)`. This eliminates the string-matching proxy used in prior work. All code is in `sok_ipl/native/pipeline_builder.py` and `scripts/run_native_benchmark.py`.

The prior version of this paper included supplemental experiments (unknown-defense settings, no-feedback settings, cross-defense transfer, threshold sweeps, and held-out defense validation) conducted under the string-matching proxy methodology. Because these proxy results *contradict* our native end-to-end findings (e.g., proxy reported Spotlighting ASR=1.00; native shows ASR=0.21), we have removed them from this revision rather than present unsupported numbers. We summarize what was removed and what remains:

a) Removed (proxy-based, not re-run natively).: (1) Settings A/B/C (unknown-defense, no-feedback, cross-defense transfer)—all used proxy ASR. (2) Threshold sweep for Task Shield and Attention Tracker—used proxy FPR/USR metrics; the native pipeline does not use the same threshold mechanism. (3) Held-out defense validation via SecAlign—used proxy ASR on Mistral-7B. (4) Query/cost analysis referencing the 3,900-trial proxy evaluation.

b) Retained (native end-to-end).: (1) Main results table (Table III): 11 defenses, 990 trials, native `security_from_traces` + `user_task.utility`. (2) P2 ablation (Section V-G, Appendix A): 270 trials, three configurations. (3) Over-blocking diagnosis (Appendix A): baseline comparison on identical task triples. (4) Statistical analysis (Appendix A): Wilson CIs from native data.

c) Planned.: We plan to re-run threshold sweeps, cross-defense transfer, and held-out validation using the native pipeline in future work. The native pipeline infrastructure

(Appendix A) supports these experiments; the constraint is API cost and time.

ACKNOWLEDGMENT

The authors thank the anonymous reviewers for their constructive feedback. Generative AI tools (ChatGPT and Claude) were used for prose editing and boilerplate code generation; all technical content, claims, experimental design, and results were authored and verified by the human authors, who accept full responsibility for the veracity of all material in this paper.

REFERENCES

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (AISec '23)*, 2023.

[2] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," arXiv:2403.14720, 2024.

[3] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "Struq: Defending against prompt injection with structured queries," arXiv:2402.06363, 2024.

[4] F. Jia, T. Wu, Q. Xin, and A. Squicciarini, "The task shield: Enforcing task alignment to defend against indirect prompt injection in llm agents," pp. 29 680–29 697, 2025.

[5] K.-H. Hung, C. Ko, A. Rawat, I. Chung, W. H. Hsu, and P. Chen, "Attention tracker: Detecting prompt injection attacks in llms," pp. 2309–2322, 2025.

[6] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, "Isolategpt: An execution isolation architecture for llm-based systems," 2025.

[7] E. Li, T. Mallick, E. Rose, W. Robertson, A. Oprea, and C. Nita-Rotaru, "Ace: A security architecture for llm-integrated app systems," in *Network and Distributed System Security (NDSS) Symposium 2026*, 2025.

[8] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "Secalign: Defending against prompt injection with preference optimization," 2024.

[9] Q. Zhan, R. Fang, H. S. Panchal, and D. K. 0001, "Adaptive attacks break defenses against indirect prompt injection attacks on llm agents," in *NAACL*, 2025.

[10] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents," arXiv:2406.13352, 2024.

[11] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents," in *Network and Distributed System Security (NDSS) Symposium 2025*, 2025.

[12] Z. Wang, N. Nagaraja, L. Zhang, H. Bahsi, P. Patil, and P. Liu, "To protect the llm agent against the prompt injection attack with polymorphic prompt," arXiv:2506.05739, 2025.

[13] R. Zhang, D. Sullivan, K. Jackson, P. Xie, and M. Chen, "Defense against prompt injection attacks via mixture of encodings," arXiv:2504.07467, 2025.

[14] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji, "Ipiguard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents," arXiv:2508.15310, 2025.

[15] J. Wang, F. Wu, W. Li, J. Pan, E. Suh, Z. M. Mao, M. Chen, and C. Xiao, "Fath: Authentication-based test-time defense against indirect prompt injection attacks," arXiv:2410.21492, 2024.

[16] D. Cheng and W.-K. Tsao, "Agent privilege separation in openclaw: A structural defense against prompt injection," arXiv:2603.13424, 2026.

[17] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, "Melon: Provable defense against indirect prompt injection attacks in ai agents," arXiv:2502.05174, 2025.

[18] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents," pp. 10 471–10 506, 2024.

[19] X. Wang, J. Bloch, Z. Shao, Y. Hu, S. Zhou, and N. Z. Gong, "Webinject: Prompt injection attack to web agents," The 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025), 2025.

[20] Z. Wang, V. Siu, Z. Ye, T. Shi, Y. Nie, X. Zhao, C. Wang, W. Guo, and D. Song, "Agentvigil: Generic black-box red-teaming for indirect prompt injection against llm agents," arXiv:2505.05849, 2025.

[21] G. Syros, E. Rose, B. Grinstead, C. Kerschbaumer, W. Robertson, C. Nita-Rotaru, and A. Oprea, "Muzzle: Adaptive agentic red-teaming of web agents against indirect prompt injection attacks," arXiv:2602.09222, 2026.

[22] Y. Chen, H. Li, Y. Sui, Y. Song, and B. Hooi, "Backdoor-powered prompt injection attacks nullify defense methods," arXiv:2510.03705, 2025.

[23] S. Gulyamov, A. Rodionov, R. Khursanov, K. Mekhmonov, D. Babaev, and A. Rakhimjonov, "Prompt injection attacks in large language models and ai agent systems: A comprehensive review of vulnerabilities, attack vectors, and defense mechanisms," *Information*, vol. 17, no. 1, p. 54, 2026.

[24] T. Geng, Z. Xu, Y. Qu, and W. E. Wong, "Prompt injection attacks on large language models: A survey of attack methods, root causes, and defense strategies," *Computers, Materials & Continua*, vol. 87, no. 1, pp. 1–10, 2025.

[25] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, "Prompt injection attack to tool selection in llm agents," arXiv:2504.19793, 2025.